

```
In [ ]: import random

import marimo as mo
from ortools.sat.python import cp_model
```

Trabalho Prático: Sudoku Genérico como CSP

A biblioteca de modelação escolhida foi o **OR-Tools**, com o resolvidor **CP-SAT**, que é a sugestão da disciplina. O Sudoku é um CSP com domínios finitos e pequenos e um único tipo de restrição ("todos diferentes"), e o CP-SAT tem essa restrição de forma nativa (`add_all_different`). Assim, cada grupo de células corresponde a exatamente uma restrição do modelo, sem ser preciso escrever as desigualdades par a par. Além disso, é um resolvidor completo: quando responde que não há solução (`INFEASIBLE`), isso é uma prova, e não apenas "não encontrei".

Um grupo genérico de células com a restrição "todos diferentes" (R1)

A classe `box` não sabe nada sobre linhas, colunas, blocos ou Sudoku: guarda apenas o dicionário v e valida cada célula que entra.

R1 (`add`): a célula (l, c) com valor val só é aceite se

$$1 \leq l \leq N \wedge 1 \leq c \leq N \wedge (val = \text{None} \vee 1 \leq val \leq N)$$

R1 (`matriz`):

$$M_{l,c} = \begin{cases} v(l, c) & \text{se } (l, c) \in B \text{ e } v(l, c) \neq \text{None} \\ 0 & \text{caso contrário} \end{cases}$$

```
In [ ]: class box:
    def __init__(self, n, celulas=None):
        self.n = n
        self.tamanho = n * n
        self.celulas = {}
        for (lin, col), val in (celulas or {}).items():
            self.add(lin, col, val)

    def add(self, lin, col, val=None):
        if not (1 <= lin <= self.tamanho and 1 <= col <= self.tamanho):
            raise ValueError(f"Célula ({lin}, {col}) fora da grelha")
        if val is not None and not (1 <= val <= self.tamanho):
            raise ValueError(f"Valor fora do intervalo [1, {self.tamanho}]")
```

```

        self.celulas[(lin, col)] = val
    return self

    def matriz(self):
        m = [[0] * self.tamanho for _ in range(self.tamanho)]
        for (lin, col), val in self.celulas.items():
            if val is not None:
                m[lin - 1][col - 1] = val
        return m

```

Um bloco $n \times n$ é um caso particular de grupo (R2)

`cube` herda de `box` e apenas escolhe as células do bloco; a validação continua a ser feita pelo `add`.

R2: para $1 \leq q_l, q_c \leq n$,

$$\text{cube}(q_l, q_c) = \{ ((q_l - 1)n + a, (q_c - 1)n + b) : 1 \leq a, b \leq n \}$$

```

In [ ]: class cube(box):
    def __init__(self, n, ql, qc):
        super().__init__(n)
        if not (1 <= ql <= n and 1 <= qc <= n):
            raise ValueError("Índices errados!")
        for lin in range((ql - 1) * n + 1, ql * n + 1):
            for col in range((qc - 1) * n + 1, qc * n + 1):
                self.add(lin, col)

```

Linhas e colunas são troços retos de células (R3)

`path` aceita os extremos em qualquer ordem: o passo s é $+1$ se o fim está depois do início e -1 se está antes.

R3: com $s = +1$ se o fim é maior ou igual ao início e $s = -1$ caso contrário,

$$\text{path}((l_1, c_1), (l_2, c_2)) = \begin{cases} \{ (l_1, c_1 + st) : 0 \leq t \leq |c_2 - c_1| \} & \text{se } l_1 = l_2 \text{ (horizontal)} \\ \{ (l_1 + st, c_1) : 0 \leq t \leq |l_2 - l_1| \} & \text{se } c_1 = c_2 \text{ (vertical)} \end{cases}$$

Se $l_1 \neq l_2$ e $c_1 \neq c_2$, o troço não é reto e é rejeitado.

```

In [ ]: class path(box):
    def __init__(self, n, inicio, fim):
        super().__init__(n)
        (lin1, col1), (lin2, col2) = inicio, fim
        if lin1 != lin2 and col1 != col2:
            raise ValueError("Apenas devemos verificar linhas e/ou colunas!")
        if lin1 == lin2:
            step = 1 if col2 >= col1 else -1
            coords = [(lin1, c) for c in range(col1, col2 + step, step)]

```

```

else:
    step = 1 if lin2 >= lin1 else -1
    coords = [(l, col1) for l in range(lin1, lin2 + step, step)]

for lin, col in coords:
    self.add(lin, col)

```

As pistas aleatórias são apenas um grupo com células fixas (R4)

R4:

$$S \subseteq G, \quad |S| = k, \quad 0 \leq k \leq N^2, \quad \forall (l, c) \in S: v(l, c) \sim \mathcal{U}\{1, \dots, N\}$$

$$i \in \{0, \dots, N^2 - 1\} \mapsto (\lfloor i/N \rfloor + 1, (i \bmod N) + 1)$$

```

In [ ]: def pistas_aleatorias(n, k=None, seed=None):
    tamanho = n * n
    if k is None:
        k = n
    if not (0 <= k <= tamanho * tamanho):
        raise ValueError(f"k tem que estar entre 0 e {tamanho * tamanho}")
    ran = random.Random(seed)
    pistas = box(n)
    for c in ran.sample(range(tamanho * tamanho), k):
        lin, col = c // tamanho + 1, c % tamanho + 1
        pistas.add(lin, col, ran.randint(1, tamanho))
    return pistas

```

O modelo trata todos os grupos da mesma forma (R5)

R5 (variáveis):

$$x_{l,c} \in \{1, \dots, N\} \quad \forall (l, c) \in G$$

R5 (restrições), para cada grupo (B, v) recebido:

$$\text{AllDifferent}(x_{l,c} : (l, c) \in B) \iff \forall p, q \in B, p \neq q : x_p \neq x_q$$

$$\forall (l, c) \in B \text{ com } v(l, c) \neq \text{None} : \quad x_{l,c} = v(l, c)$$

```

In [ ]: class Sudoku_Solver:
    def __init__(self, n):
        self.n = n
        self.tamanho = n * n
        self.modelo = cp_model.CpModel()
        self.x = {
            (lin, col): self.modelo.new_int_var(1, self.tamanho, f"x[{lin}],

```

```

{col}]]")
        for lin in range(1, self.tamanho + 1)
        for col in range(1, self.tamanho + 1)
    }
    self.estado = None

    def add(self, *grupos):
        for g in grupos:
            variaveis = [self.x[cel] for cel in g.celulas]
            if len(variaveis) > 1:
                self.modelo.add_all_different(variaveis)
            for cel, val in g.celulas.items():
                if val is not None:
                    self.modelo.add(self.x[cel] == val)
        return self

    def resolver(self, tempo_max=10.0):
        solver = cp_model.CpSolver()
        solver.parameters.max_time_in_seconds = tempo_max
        status = solver.solve(self.modelo)
        self.estado = solver.status_name(status)

        if status not in (cp_model.OPTIMAL, cp_model.FEASIBLE):
            return None

        return [
            [solver.value(self.x[(lin, col)]) for col in range(1,
self.tamanho + 1)]
            for lin in range(1, self.tamanho + 1)
        ]

```

Um Sudoku completo é a junção de linhas, colunas, blocos e pistas (R6)

O `Sudoku_Solver` não sabe o que é um Sudoku: as regras aparecem apenas porque lhe passamos os grupos certos, $3n^2 + 1$ ao todo (28 num 9×9). Não há nenhuma restrição nova.

R6 (linhas, com `path((l, 1), (l, N))`):

$$\forall l \in \{1, \dots, N\} : \quad \text{AllDifferent}(x_{l,1}, \dots, x_{l,N})$$

R6 (colunas, com `path((1, c), (N, c))`):

$$\forall c \in \{1, \dots, N\} : \quad \text{AllDifferent}(x_{1,c}, \dots, x_{N,c})$$

R6 (blocos, com `cube(ql, qc)`):

$$\forall q_l, q_c \in \{1, \dots, n\} : \quad \text{AllDifferent}(x_{(q_l-1)n+a, (q_c-1)n+b} : 1 \leq a, b \leq n)$$

R6 (pistas, com `pistas_aleatorias(n, k, seed)`):

$$\forall (l, c) \in S: x_{l,c} = v(l, c), \quad \text{AllDifferent}(x_{l,c} : (l, c) \in S)$$

```
In [ ]: def resolver_sudoku(n, k=None, seed=None):
    tamanho = n * n
    linhas = [path(n, (l, 1), (l, tamanho)) for l in range(1, tamanho + 1)]
    colunas = [path(n, (1, c), (tamanho, c)) for c in range(1, tamanho + 1)]
    blocos = [cube(n, ql, qc) for ql in range(1, n + 1) for qc in range(1, n + 1)]
    pistas = pistas_aleatorias(n, k, seed)
    return Sudoku_Solver(n).add(*linhas, *colunas, *blocos,
    pistas).resolver()
```

Representar a solução

Tabela HTML: as linhas grossas separam os blocos $n \times n$ e as pistas aparecem a amarelo. As células com valor 0 (livres, na matriz das pistas) aparecem em branco.

```
In [ ]: def mostrar_sudoku(grelha, n, pistas=None):
    tamanho = n * n
    fixas = set()
    if pistas is not None:
        fixas = {cel for cel, val in pistas.celulas.items() if val is not None}
    linhas = []
    for lin in range(1, tamanho + 1):
        celulas = []
        for col in range(1, tamanho + 1):
            estilo = "width:2.1em;height:2.1em;text-align:center;border:1px solid #9995;"
            if (lin - 1) % n == 0:
                estilo += "border-top:2.5px solid currentColor;"
            if (col - 1) % n == 0:
                estilo += "border-left:2.5px solid currentColor;"
            if lin == tamanho:
                estilo += "border-bottom:2.5px solid currentColor;"
            if col == tamanho:
                estilo += "border-right:2.5px solid currentColor;"
            if (lin, col) in fixas:
                estilo += "background:#ffd166;color:#000;font-weight:bold;"
            valor = grelha[lin - 1][col - 1] or ""
            celulas.append(f'<td style="{estilo}">{valor}</td>')
        linhas.append("<tr>" + "".join(celulas) + "</tr>")
    return mo.Html(
        '<table style="border-collapse:collapse;font-family:monospace">'
        + "".join(linhas)
        + "</table>"
    )
```

Validação

Para uma grelha resolvida g , verifica-se o que o enunciado pede. A validação percorre linhas, colunas e blocos diretamente, sem usar `path` nem `cube`: se usasse os mesmos grupos que o modelo, um erro nessas classes passaria despercebido. Como cada conjunto tem N valores, ser igual a $\{1, \dots, N\}$ garante que não há repetições.

$$\forall l: \{g_{l,c} : 1 \leq c \leq N\} = \{1, \dots, N\} \quad \forall c: \{g_{l,c} : 1 \leq l \leq N\} = \{1, \dots, N\}$$

$$\forall q_l, q_c: \{g_{(q_l-1)n+a, (q_c-1)n+b} : 1 \leq a, b \leq n\} = \{1, \dots, N\} \quad \forall (l, c) \in S: g_{l,c} = v(l$$

A função devolve a lista de erros encontrados (lista vazia = grelha válida).

```
In [ ]: def validar(grelha, n, pistas=None):
    tamanho = n * n
    todos = set(range(1, tamanho + 1))
    erros = []
    for lin in range(1, tamanho + 1):
        if {grelha[lin - 1][col - 1] for col in range(1, tamanho + 1)} !=
    todos:
        erros.append(f"linha {lin}")
    for col in range(1, tamanho + 1):
        if {grelha[lin - 1][col - 1] for lin in range(1, tamanho + 1)} !=
    todos:
        erros.append(f"coluna {col}")
    for ql in range(1, n + 1):
        for qc in range(1, n + 1):
            bloco = {
                grelha[(ql - 1) * n + a - 1][(qc - 1) * n + b - 1]
                for a in range(1, n + 1)
                for b in range(1, n + 1)
            }
            if bloco != todos:
                erros.append(f"bloco ({ql}, {qc})")
    if pistas is not None:
        for (lin, col), val in pistas.celulas.items():
            if val is not None and grelha[lin - 1][col - 1] != val:
                erros.append(f"pista ({lin}, {col}) = {val} alterada")
    return erros
```

Exemplos

Fluxo completo pedido no enunciado: gerar pistas aleatórias → montar linhas + colunas + blocos + pistas → resolver → validar, com $n = 3$ e $n = 2$. As sementes estão fixas para os exemplos serem reproduzíveis. Como a mesma `seed` gera sempre as mesmas pistas, voltamos a gerá-las para as mostrar e validar.

```
In [ ]: def mostrar_exemplo(n, seed, k=None):
    pistas = pistas_aleatorias(n, k, seed) # as mesmas pistas que o R6 usa
    grelha = resolver_sudoku(n, k, seed)
    puzzle = mo.vstack([mo.md("**Puzzle**"),
    mostrar_sudoku(pistas.matriz(), n, pistas)])
    if grelha is None:
```

```

        return mo.vstack([
            mo.md(f"n = {n}, seed = {seed}: ⚠️ **sem solução** (o
resolver devolveu `None`)",
                puzzle,
            ])
        erros = validar(grelha, n, pistas)
        resultado = "✅ grelha válida" if not erros else "❌ " + ";
".join(erros)
        return mo.vstack([
            mo.md(f"n = {n} ({n * n}x{n * n}), seed = {seed},
{len(pistas.celulas)} pistas · {resultado}"),
            mo.hstack(
                [puzzle, mo.vstack([mo.md("***Solução***"),
mostrar_sudoku(grelha, n, pistas)])],
                justify="start",
                gap=2,
            ),
        ])
    )

```

Exemplo 1: Sudoku clássico 9×9 ($n = 3$)

In []: `mostrar_exemplo(3, seed=1)`

$n = 3$ (9×9), seed = 1, 3 pistas · ✅ grelha válida

Puzzle

								8
								5
2								

Solução

4	2	5	3	1	6	7	9	8
8	3	6	2	7	9	1	4	5
1	7	9	5	4	8	3	2	6
5	6	4	1	2	3	8	7	9
3	1	2	8	9	7	5	6	4
9	8	7	6	5	4	2	1	3
6	5	1	4	3	2	9	8	7
7	4	3	9	8	1	6	5	2
2	9	8	7	6	5	4	3	1

Exemplo 2: Sudoku 4×4 ($n = 2$)

O mesmo código, sem alterações, com outro valor de n : nada está fixo a 9×9 .

```
In [ ]: mostrar_exemplo(2, seed=1)
```

n = 2 (4×4), seed = 1, 2 pistas ·  grelha válida

Puzzle

1			
	3		

Solução

3	4	1	2
1	2	3	4
2	3	4	1
4	1	2	3

Exemplo 3: puzzle sem solução

Com `seed = 6` saem duas pistas com o valor 1, em (2, 2) e (7, 9). Num Sudoku isto seria válido, porque estão em linhas, colunas e blocos diferentes. Mas o grupo das pistas também tem a restrição "todos diferentes" (R5), por isso o resolvidor prova que não há solução e o programa reporta o insucesso.

```
In [ ]: mostrar_exemplo(3, seed=6)
```

n = 3, seed = 6:  **sem solução** (o resolvidor devolveu `None`)

Puzzle

	1							
								1
	5							

Ferramentas de Apoio

- [Documentação do CP-SAT \(OR-Tools\)](#)
- [Documentação do marimo](#)
- <https://ericpony.github.io/z3py-tutorial/guide-examples.htm>

Ferramentas de LLM

- [Claude](#)